

Aufg. 1

In Aufgabe 1 wurde ein vollständiges Zoo-System entwickelt, das auf einer klar strukturierten Objektorientierung basiert. Zunächst wurde eine Tierhierarchie aufgebaut, in der das Interface `Animal` als gemeinsame Grundlage diente und durch abstrakte Oberklassen wie `Mammal`, `Fish`, `Reptile` und `Bird` erweitert wurde. Darauf aufbauend wurden konkrete Tierklassen wie `Trout`, `Salmon`, `Gorilla`, `Mouse`, `HouseCat`, `Snake` und `Lizard` implementiert, sodass der Zoo verschiedene Tierarten verwalten kann.

Anschließend wurde ein generisches Gehege `Enclosure<T extends Animal>` erstellt. Dabei wurde sichergestellt, dass jedes Gehege nur Tiere eines bestimmten Typs aufnehmen kann. Durch den Einsatz von Generics wurde erreicht, dass falsche Tier-Gehege-Kombinationen bereits zur Compile-Zeit verhindert werden. Auf Basis dieses generischen Geheges wurden spezialisierte Gehege wie `Aquarium`, `Terrarium`, `MammalHouse` und `CatHouse` implementiert, die jeweils nur passende Tierarten zulassen.

Daraufhin wurde die zentrale Verwaltungsklasse `Zoo` entwickelt. Sie verwaltet alle Gehege, ermöglicht Abfragen über sämtliche Tiere und stellt Funktionen wie das Finden von Gehegen, das Ermitteln aller Tiere oder das Erstellen von Statistiken bereit. Für diese Abfragen wurden Java-Streams eingesetzt, um Daten aus allen Gehegen effizient zusammenzuführen und zu filtern. Die Zoo-Klasse wurde außerdem so vorbereitet, dass sie systematisches Logging unterstützt, indem ein Logger integriert wurde, der in späteren Aufgaben erweitert wird.

Damit wurde in Aufgabe 1 ein vollständiges, typsicheres und erweiterbares Zoo-Management-System geschaffen, das sowohl objektorientierte Prinzipien als auch moderne Java-Features wie Generics und Streams nutzt.

Aufg. 2

In Aufgabe 2 wurde das bestehende Zoo-System um ein systematisches Logging erweitert. Dazu wurde in der Zoo-Klasse ein zentraler Logger eingebunden, der alle wichtigen Methodenaufrufe und Systemzustände protokolliert. Der Logger ersetzt einfache Konsolenausgaben und ermöglicht eine strukturierte, filterbare und professionell nutzbare Protokollierung. Für jede Methode wurden passende Log-Level eingesetzt: INFO für reguläre Abläufe, FINE für detaillierte Statusmeldungen und WARNING für unerwartete, aber nicht kritische Situationen wie das Nichtfinden eines Geheges. Zusätzlich wurde eine Demo-Main erstellt, die zeigt, wie das Log-Level zur Laufzeit angepasst werden kann, sodass sowohl grobe als auch sehr detaillierte Informationen sichtbar werden. Durch diese Erweiterung ist das Zoo-System deutlich besser nachvollziehbar, leichter zu debuggen und entspricht den Anforderungen eines realistischen Anwendungsszenarios.

Aufg.3

Generics

Wo helfen Ihnen die Generics im Zoo-Szenario, Fehler bereits zur Compile-Zeit zu vermeiden?

- Typsicherheit bei Gehegen: Ein `Enclosure<T>` kann nur Tiere vom Typ `T` aufnehmen.
- Compiler verhindert falsche Kombinationen (z. B. Fisch ins Terrarium).
- Keine Casts nötig → weniger Laufzeitfehler.
- API ist klarer: Methoden signalisieren, welche Tierarten erlaubt sind.

Implementierung

```
class Aquarium extends Enclosure<Fish>
```

- → `aquarium.add(new Eagle("Horst"))`; führt zu Compile-Fehler, nicht erst zur Laufzeit.

Logging

Warum ist systematisches Logging mit einem Logger und Log-Levels für ein Zoo-Management-System sinnvoller als Ausgaben mit `IO.println`?

- Log-Level steuerbar (INFO, WARNING, FINE ...) → gezielte Analyse.
- Logs können gefiltert, gespeichert, weitergeleitet werden.
- `println` ist unstrukturiert und nicht abschaltbar.
- Ermöglicht Debugging ohne Codeänderungen (Level umschalten statt Code editieren).

In welchen Situationen würden Sie in Ihrem System die Log-Level INFO, WARNING und ggf. SEVERE verwenden?

- INFO
 - o Normale Betriebsinformationen
 - o Methodenstarts (`addEnclosure`, `getAllAnimals`, `summary`)
 - o Wichtige, aber erwartbare Ereignisse
- WARNING
 - o Nicht-kritische Fehler
 - o Gehege nicht gefunden
 - o Tier nicht vorhanden
 - o Ungültige Eingaben

- SEVERE
 - o Kritische Systemfehler
 - o Inkonsistenzen im Datenmodell
 - o Zoo-Struktur beschädigt
 - o Exceptions, die nicht abgefangen werden können

Streams

Wo haben Ihnen Streams im Vergleich zu klassischen Schleifen beim Formulieren Ihrer Abfragen geholfen?

Kürzere, klarere Abfragen über alle Gehege:

- `flatMap(e -> e.getInhabitants().stream())`

Filterlogik elegant:

- `.filter(a -> a instanceof Mammal)`

Statistiken einfach:

- `groupingBy(Animal::getClass, counting())`
- Keine manuellen Schleifen, keine temporären Listen.

Wo wurde es eher unübersichtlich?

- Wenn die Typen durch Generics kompliziert wurden (z. B. `? extends Animal`).
- Wenn der Rückgabetypp sehr verschachtelt war (z. B. eine Map mit generischen Klassen).
- Wenn viele Stream-Schritte hintereinander ausgeführt wurden (z. B. `flatMap` → `filter` → `map` → `collect`).
- Wenn man zwischendurch Debug-Ausgaben gebraucht hätte, weil Streams dafür weniger geeignet sind als Schleifen.